



Computer Games Development CW208
TDD
Year IV

Emily Breen
C00288668
11/04/2026

Technical Overview

Can I Hinder? is a crowd- interactive game built as a distributed system, Instead of it being a single application it is made up of several connected components, each responsible for a different part of the experience.

At its core, the system combines a real-time C++ game client with web technologies that allow spectators to interact with gameplay as it happens. These interactions are handled through a WebSocket relay server, supported by a REST API for authentication and a Progressive Web App (PWA) for user interaction.

The aim of this document is to explain how these components work together, including the system architecture, data flow, class design and key technical decisions.

Technology Stack

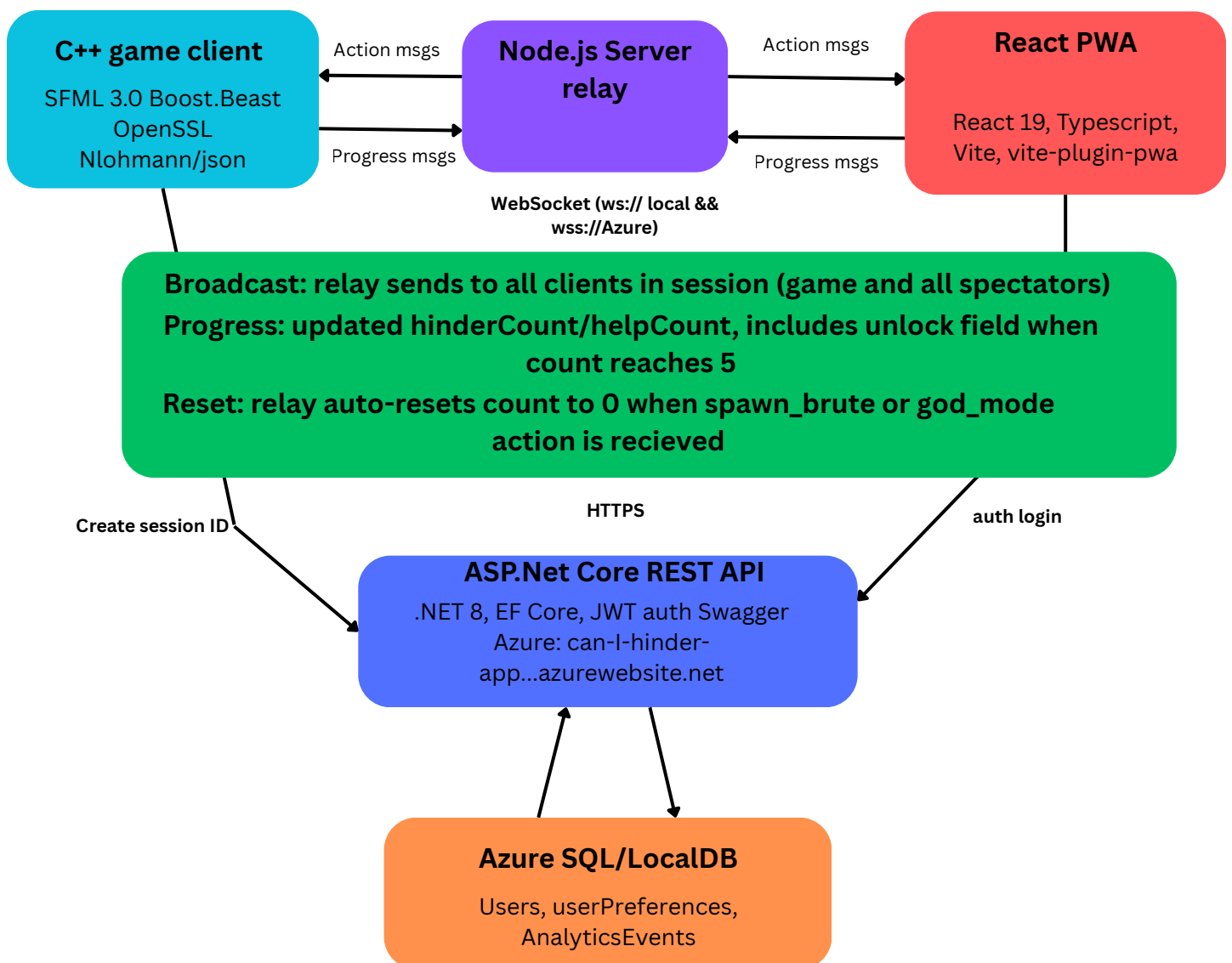
The project uses a mix of native and web technologies to balance performance and accessibility. The game client is written in C++ using SFML while the surrounding systems use modern web frameworks

	Technology		Component	Purpose
	C++/SFML L 3.0		Game Client	Rendering, input handling, audio, game loop
	Boost.Asio /Beast		Game Client	Asynchronous networking (HTTP + WebSocket)
	OpenSSL		Game Client	Secure WebSocket communication (wss://)
	Node.js / ws		Relay Server	Real-time WebSocket communication
	ASP.Net Core 8		REST API	Authentication and session management
	Entity FrameWor k Core		REST API	Database ORM
	React /Typescript		PWA	Spectator interface
	Vite /PWA Plugin		PWA	Build tooling and offline support

System Architecture

Component Overview

The system is designed around a central communication model where all real-time interaction passes through a single relay server. This approach avoids direct coupling between clients and ensures that all clients remain synchronised. The game client and PWA both connect to this relay using WebSockets, while the REST API operates independently to handle authentication and session creation. This separation of components improves maintainability and scalability, which is a key principle in modern game system design.



WebSocket Message Protocol

Communication between the spectator interface and the game client is handled using structured JSON messages sent over WebSockets. This allows persistent, bidirectional communication between clients, reducing latency compared to traditional HTTP-based approaches.

Spectator to game messages follow this JSON schema:

```
Message from server: {"user":"Diddly widdly","action":"hinder","effect":"steal_power"}
```

Server to all clients progress messages (broadcast by relay after every Spectator action):

```
Help count: 5 pwa-client.tsx:53  
Unlocked: god_mode pwa-client.tsx:56  
Message from server: {"user":"Diddly widdly","action":"help","effect":"shield_player"} pwa-client.tsx:40  
Sent: ▶ {user: 'Diddly widdly', action: 'help', effect: 'god_mode'} pwa-client.tsx:85
```

```
Hinder count: 5 pwa-client.tsx:52  
Help count: undefined pwa-client.tsx:53  
Unlocked: spawn_brute pwa-client.tsx:56  
Message from server: {"user":"Diddly widdly","action":"hinder","effect":"spawn_enemy"} pwa-client.tsx:40  
Sent: ▶ {user: 'Diddly widdly', action: 'hinder', effect: 'spawn_brute'} pwa-client.tsx:85
```

Server to all clients reset messages (broadcast when spawn_brute or god_mode is used, resetting the session count so the unlock must be earned again)

```
Message from server: {"type":"reset","action":"help"} pwa-client.tsx:40
```

```
Message from server: {"type":"reset","action":"hinder"} pwa-client.tsx:40
```

C++ Game Client - Class Design

Class Hierarchy

The game client is structured using an object-oriented design, where each class is responsible for a clearly defined part of the system. This approach allows gameplay logic, rendering, input handling, and networking to be developed independently while remaining tightly integrated within the main game loop. The following table summarises the key classes used within the game client and their responsibilities.

Class /File	Inherits/Uses	Responsibility
Entity		Abstract base class. Holds sf::Sprite and sf::Texture. Declares pure virtual update(float dt) and virtual draw(). Provides static rectsIntersect() utility.
Player	Entity	Player movement (8-directional, keyboard and Xbox controller), melee combat with directional hitbox, animation state machine, buff/debuff targets, victory pose, battle mode toggle.
NPC	Entity	Enemy entity. Owns one AIBehaviour instance. Manages animation states, health bar, attack range/cooldown, loot drop flag, and boss mode switching (MapIdle / Battle).
Game		Top-level orchestrator. Owns the game loop (processEvents / update / render), all subsystems, menu state machine, WebSocket callbacks, and all gameplay effect application logic.
AIBehaviour		Stateless steering engine. Implements Seek, Arrive, Pursue, Wander, Flee, wall avoidance, and NPC separation. Called per-frame by NPC::computeAIMovement().
AnimationHandler		Frame-sequence manager. Maps (PlayerState, Direction) pairs to sprite sheet rects. Advances frames by elapsed time and applies to sf::Sprite.
Map	sf::Drawable	Wraps TMX Lite. Loads .tmx maps, renders layered tile batches, extracts collision rects and DoorData. Provides typed spawn vectors: key, health, speed, power potion positions.

Class /File	Inherits/Users	Responsibility
HUD		Renders health bar, session code text, 6-slot hotbar (slots 0-2 keys, 3-5 potions), and chat box (max 10 messages, 6s TTL). Colour-coded help (green) / hinder (red).
Camera		Manages sf::View. Follows player position smoothly. Clamps view to map bounds. Handles window resize events.
InputHandler		Abstracts keyboard and Xbox controller input (sf::Joystick). Controller auto-detected at startup. Provides movement vector, attack/action flags, and menu navigation inputs.
WebsocketClient		Async WebSocket client using Boost.Beast. Runs on a dedicated std::thread. Supports ws:// (local) and wss:// (Azure TLS). Mirrors server-side m_hinderCount and m_helpCount locally.
Audio		Wraps sf::Music and sf::SoundBuffer. Named play/stop methods for per-state background music and SFX playback.
Items	Entity	Collectible world item (healthPotion, speedPotion, powerPotion, Key). Bobs vertically via timer. Removed on player overlap; triggers HUD update and hotbar insertion.
FinalLevel		Manages the turn-based boss battle: Dialogue state machine, TurnState machine (PLAYER_CHOICE, PLAYER_ACTION, ENEMY_ACTION, WIN, LOSE), spell casting, MP management, floating DamageText, spell banners, and fade transitions.
Effects		Animated visual effect. 17 EffectTypes. Supports projectile velocity, player-follow attachment, lifetime callback (setOnFinish), and spawn-trigger handshake (shouldSpawn / markAsSpawned).
Obstacles		Spawned map hazard for drop_trap. Rendered at placement position. Expires after 5 seconds via sf::Clock.
MathUtils		Vector utility helpers: normalise, magnitude, distance. Used throughout project

Key Enums

MenuState	MAIN_MENU, SETTINGS, GAMEPLAY, BOSS_DIALOGUE, BOSS_BATTLE, PAUSE, GAME_OVER, GAME_WON
EnemyType	Skeleton, Goblin, Brute, Boss, None
AIBehaviour::Mode	Seek, Arrive, Pursue, Wander, Flee, None
BossMode	MapIdle, Battle: switches NPC animation set and AI behaviour for the boss
PlayerState	IDLE, WALK, ATTACK, HURT, DEAD: drives animation selection for both Player and NPC
MagicType	Fireball, Thunder, Heal, Ice, Gravity: used by FinalLevel for spell selection
TurnState	PLAYER_CHOICE, PLAYER_ACTION, ENEMY_ACTION, WIN, LOSE: FinalLevel battle state machine
EffectType	SpawnEnemy, Fireball, FireHit, Thunder, ThunderHit, Ice, IceHit, Gravity, GravityHit, Healing, SpeedBoost, PowerBoost, StealPower, SlowPlayer, SheildPlayer, GodMode, SpawnTrap
ItemType	healthPotion, speedPotion, powerPotion, Key
DialogueState	NONE, ACTIVE, FINISHED: controls pre-battle boss dialogue sequence

AI Behaviour Design

Enemy behaviour is implemented using a steering-based system, where movement is calculated dynamically based on the player's position and the surrounding environment. Steering behaviours allow agents to react naturally rather than following predefined paths.

Behaviour Descriptions

Seek	Moves the enemy directly toward the target at maximum speed. The steering force adjusts the current movement so the enemy gradually aligns with the desired direction.
Arrive	Similar to Seek, but the enemy slows down as it approaches the target, coming to a smooth stop once it gets close enough.
Pursue	Predicts where the player is moving to and heads toward that future position instead of their current location. This allows enemies to intercept the player more effectively
Wander	Moves the enemy in a random but natural way by slightly changing its direction over time. This creates smooth, unpredictable movement instead of sharp or erratic turn.
Flee	Moves the enemy away from the target at maximum speed, creating distance when the player gets too close.

Wall Avoidance

After the main movement direction is calculated, the game checks if the enemy is about to hit a wall. If it is, the part of the movement that would push it into the wall is removed. This makes the enemy slide along the wall instead of getting stuck or passing through it. This behaviour is used for all enemies except the boss, which stays in a fixed position during battle.

NPC Seperation

To stop enemies from crowding together, the game checks for other nearby NPCs within a certain distance. It then pushes the enemy slightly away from them, creating some space between each one. This adjustment is combined with the normal movement, helping enemies spread out more naturally instead of bunching up.

Enemy AI Modes in Practice

Skeleton/Goblin (idle)	Wander - randomised patrolling until player enters detection radius
Skeleton/Goblin (chase)	Pursue - predicts player position and intercepts, wall avoidance and seperation active
Brute	Pursue mode - same as Skeleton and Goblin but slower base speed with higher damage
Boss (Map 4)	MapIdle BossMode - scripted idle animations. AI isnt active
Boss (battle)	Battle BossMode - attack animation is controlled by FinalLevel Turnstate, no steering used

WebSocket Client Design

Threading Model

The WebSocket system is responsible for enabling real-time communication between the game client and spectators. To ensure that gameplay remains responsive, all networking is handled asynchronously on a separate thread, allowing the main game loop to continue uninterrupted. The relay server processes incoming messages and manages shared session state, ensuring that all clients receive consistent updates.

Session Creation

When the game is initialised, it requests a new session from the backend API. The returned code is then used when connecting to the WebSocket server, allowing spectators to join the correct game instance.

Progress Tracking and Reset

The progress and unlock system lives entirely in the relay server. When `readLoop()` receives a progress message it updates local mirrors `m_hinderCount` and `m_helpCount` for logging. When it receives a reset message it clears those mirrors. All count incrementing, unlock detection, count resetting and broadcasting is handled by the relay when it processes incoming action messages

WebSocket Relay Design (Node.js)

Session Management

The relay server runs a single WebSocket server that handles all incoming connections. Each game session keeps track of the connected clients, along with shared state such as the help and hinder counts and whether special actions have been unlocked. When a new session is created, these values are reset and managed centrally by the server, ensuring that all connected clients remain synchronised

Message Processing and Fan Out

The relay processes each incoming message before broadcasting it to all clients. The main behaviours are as follows:

- When `spawn_brute` is received, the hinder counter is reset and a reset message is broadcast, followed by the action itself.

- When god_mode is received, the same process is applied to the help counter.
- Standard hinder actions increase the hinder count and trigger a progress update.
- Standard help actions increase the help count and follow the same logic.

When a counter reaches its threshold, a special action is unlocked and included in the progress update. All updates are then broadcast to every connected client in the session.

Both the PWA and the game client receive the same broadcasts and update their local state accordingly.

Session Cleanup

On disconnect, the client is removed from its session set, if the set becomes empty the session entry is deleted from the Map, this helps keep memory clean across many potential short lived sessions.

REST API (ASP .NET Core 8)

The REST API is built using ASP.NET Core, which provides a structured framework for handling authentication and backend services.

Endpoints

Method	Route	Auth	Description
POST	/api/auth/register	None	Register new user - username, email, and hashed password stored via EF Core
POST	/api/auth/login	None	Authenticate and return a signed JWT Bearer token
POST	/api/auth/forgot-password	None	Send password reset email and a tokenised link
POST	/api/auth/reset-password	None	Reset password using token from the email
POST	/api/auth/create-session	None	Generate and return a 6 character alphanumeric session code
GET	/api/auth/user/profile	JWT Bearer	Return the authenticated user's profile data

Session Code Generation

Session codes are six characters long and are generated using a restricted character set. Visually similar characters such as “0”, “O”, “1”, and “L” are excluded to reduce confusion when sharing codes between users.

Data Model

User	Id, Username, PasswordHash, Email, ResetToken, ResetToken Expiry
------	--

PWA (React /Typescript)

The PWA is built using React, which supports the development of responsive and interactive user interfaces. It provides the interface for spectators to join a session and interact with the game, and is designed to be simple and accessible, allowing users to quickly connect and participate from any device.

Routing

React Router 7 with a ProtectedRoute wrapper in main.tsx. Any unauthenticated users are redirected to /login before accessing the spectator control page.

WebSocket Connection

pwa-client.tsx manages a single WebSocket instance per session. The PWA receives the progress messages from the server to enable the unlock button when it is present in the message, and then disables it when the reset message is recieved.

Radial Menu

The two root modes Help/Hinder each expose four standard action buttons. The unlock button (god_mode/spawn_brute) is shown only when the unlock field is present and is hidden again in the radial menu once the action has been used.

Rate limiting

rate-limit.tsx tracks the last fired timestamp per action. If the cooldown has not elapsed the button is rendered disabled, this is to help prevent action spamming.

PWA Installation

vite-plugin-pwa generates a Web App manifest and Workbox service worker, this allows the app to be installed to the home screen on IOS, Android and desktop devices, it also caches static assets for faster loading.

Conclusion

In conclusion, Can I Hinder? demonstrates how a distributed system can extend a traditional single-player game into a real-time interactive experience. By separating gameplay, networking, and user interaction into distinct components, the system is able to maintain performance while supporting dynamic audience participation.

The project highlights how combining a high-performance game client with modern web technologies can create more engaging and interactive gameplay systems.